

Grundlagen von Datenbanken

Datenbanken

Lernzettel

Kompakte Zusammenfassung der Vorlesungsinhalte für die Prüfungsvorbereitung.

Autor Levin Rüßmann
Matrikelnummer 838791
Studiengang Informatik B.Sc.
Semester Sommersemester 2026
Datum 28. Juni 2026

Inhaltsverzeichnis

1	Datenbanken — Grundlagen	2
1.1	Datenbank – Definition	2
1.2	Datenbank-Management-System – DBMS	2
1.2.1	Anforderungen an ein DBMS	2
1.2.2	Aufbau eines DBMS	3
2	Datenbankmodelle	4
2.1	Relationales Datenbankmodell	4
2.2	Hierarchisches Datenbankmodell	5
2.3	Netzwerkmodell	6
2.4	Objektorientiertes Datenbankmodell	7
3	Datenverteilung	8
3.1	Gründe für die Datenverteilung	8
4	Datenbanken in verteilten Systemen	9
4.1	CAP-Theorem	9
4.2	Zentrale Datenbank	10
4.3	Replikation	10
4.3.1	Arten	10
4.3.2	Modus	11
4.4	Sharding	11
4.5	Caching	12
4.6	Map/Reduce	12
5	Managementunterstützungssysteme	13
5.1	Data Warehouse	13
5.2	Data Mart	14
5.3	Online Analytical Processing – OLAP	15
5.4	Data Mining	15
6	NoSQL	16
6.1	Modelle	16
6.1.1	Key-Value	16
6.1.2	Document Store	16
6.1.3	Column Store	16
6.1.4	Graphdatenbank	16
7	Datenbank-Architektur	17
7.1	ANSI-3-Ebenen-Modell	17
7.1.1	Interne Ebene	17
7.1.2	Konzeptionelle Ebene	17
7.1.3	Externe Ebene	18
7.2	Datenbank-Entwurfsphasen	19
7.2.1	Planungsphase	19
7.2.2	Definitionsphase	20
7.2.3	Entwurfsphase	20
7.2.4	Implementierungsphase	20
7.2.5	Abnahme- und Einführungsphase	20
7.2.6	Wartungs- und Pflegephase	20

1 Datenbanken — Grundlagen

1.1 Datenbank – Definition

Eine Datenbank speichert eine Vielzahl von Daten nach einem bestimmten Datenbankmodell. Im Gegensatz zu einem einfachen Verzeichnis werden die Daten strukturiert und redundanzfrei gespeichert. Außerdem ermöglicht eine Datenbank die dauerhafte (persistente) Speicherung von Daten.

1.2 Datenbank-Management-System – DBMS

Eine Datenbank wird immer durch ein Datenbank-Management-System (DBMS) verwaltet. Beispiele dafür sind MySQL, PostgreSQL oder Microsoft SQL Server. Das DBMS ist die Schnittstelle zur Datenbank – egal ob per CLI oder über eine GUI. Außerdem ermöglicht ein DBMS den Mehrnutzerbetrieb.

Vorteile

- Redundanzfreiheit – Daten werden nur einmal gespeichert
- Flexibilität – Struktur lässt sich nachträglich anpassen
- Physische Datenunabhängigkeit – Anwendung ist von der Speicherform getrennt
- Mehrnutzerbetrieb – mehrere Nutzer können gleichzeitig zugreifen
- Datenintegrität – Konsistenz wird durch das DBMS sichergestellt
- Recovery – Wiederherstellung nach einem Fehler oder Absturz
- Zugriffskontrolle – Rechte können nutzerspezifisch vergeben werden

Nachteile

- Ressourcenintensiv – benötigt mehr Rechenleistung und Speicher
- Komplexität – Einrichtung und Administration erfordern Fachwissen
- Kosten – Anschaffung, Lizenzen und Wartung verursachen laufende Ausgaben
- Schulungsaufwand – Mitarbeiter müssen im Umgang mit dem DBMS geschult werden
- Single Point of Failure – fällt das DBMS aus, sind alle Daten nicht verfügbar

1.2.1 Anforderungen an ein DBMS

1. **Redundanzabbau:** Durch Normalisierung verwaltet das DBMS die Daten so, dass sie möglichst wenig redundant gespeichert werden.
2. **Datenintegrität:** Die gespeicherten Daten müssen korrekt und zuverlässig sein.
3. **Effizienz und Performance:** Daten sollen schnell abgefragt werden können.
4. **Benutzerfreundlichkeit:** Tabellen sollten verständlich benannt sein, damit Daten leicht zu finden sind und in sinnvollen Datentypen vorliegen.
5. **Mehrfachzugriff:** Mehrere Nutzer sollen gleichzeitig auf die Datenbank zugreifen können.
6. **Datenschutz:** Daten werden durch Nutzerrollen und Anonymisierung geschützt.
7. **Datensicherheit:** Daten werden durch Backups gesichert und geschützt.
8. **Datenunabhängigkeit:** Grad, in dem Nutzer und Anwendung unabhängig von der

konkreten Speicherform auf das DBMS zugreifen können.

1.2.2 Aufbau eines DBMS

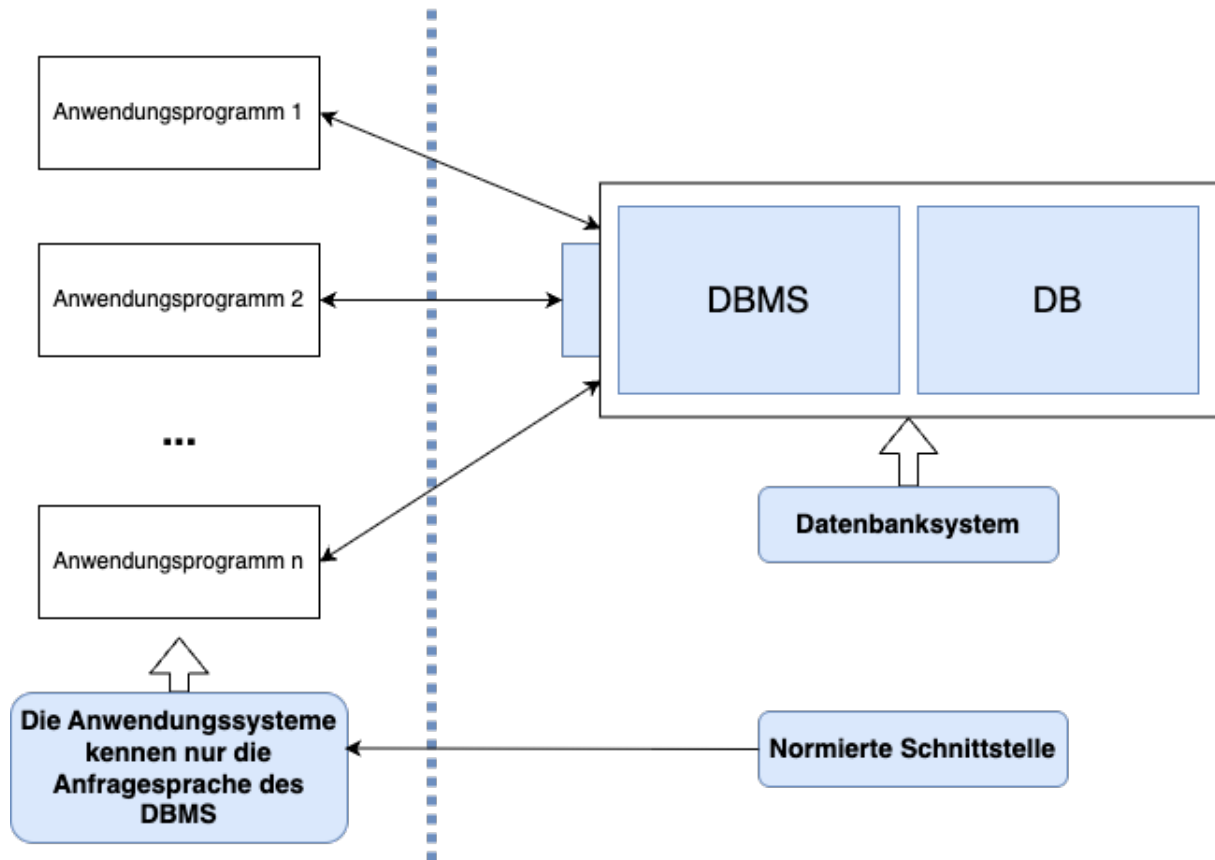


Abbildung 1. Aufbau eines DBMS

Auch bei dem Aufbau kann man Elemente wie den Mehrnutzerzugriff erkennen. Auch die Datenunabhängigkeit wird gezeigt, da die Anwendung nur die Anfragesprache kennt, aber nicht die Datenbank. Die Anfragesprache ist meist ein SQL-Dialekt.

2 Datenbankmodelle

Datenbankmodelle entscheiden, wie Daten gespeichert und wie sie abgerufen werden. Jedes der folgenden Datenbankmodelle hat einen eigenen sinnvollen Use Case. Keins der Modelle ist schlecht, trotzdem hat sich heutzutage das relationale Datenbankmodell durchgesetzt.

2.1 Relationales Datenbankmodell

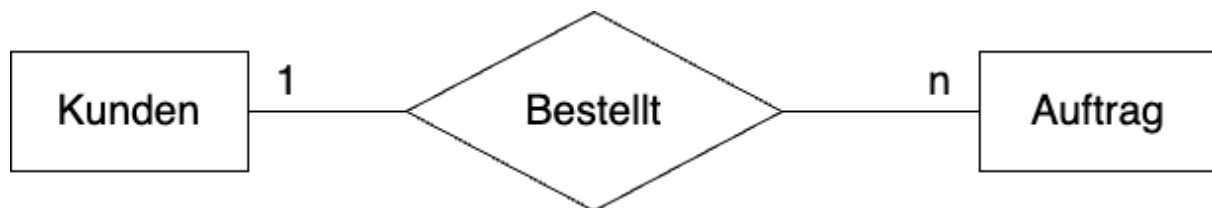


Abbildung 2. Beispiel: relationales Datenbankmodell

Ein relationales Datenbankmodell besteht aus *Relationen*, also Tabellen. Jede Zeile ist ein *Tupel*. Ein Tupel ist eine Liste mit fester Länge, die nicht mehr geändert werden kann. Im Fall des relationalen Datenbankmodells bestehen die Tupel aus einer Reihe von Attributen. Das Schema einer Relation legt dabei die Anzahl und den Typ der Attribute fest. Ein Datensatz in der Tabelle wird `Record` genannt.

Vorteile

- Große Verbreitung
- Einfach und flexibel

Nachteile

- Aufwendige Segmentierung
- Künstliche Schlüsselattribute wie IDs
- Anspruchsvolle Komposition

2.2 Hierarchisches Datenbankmodell

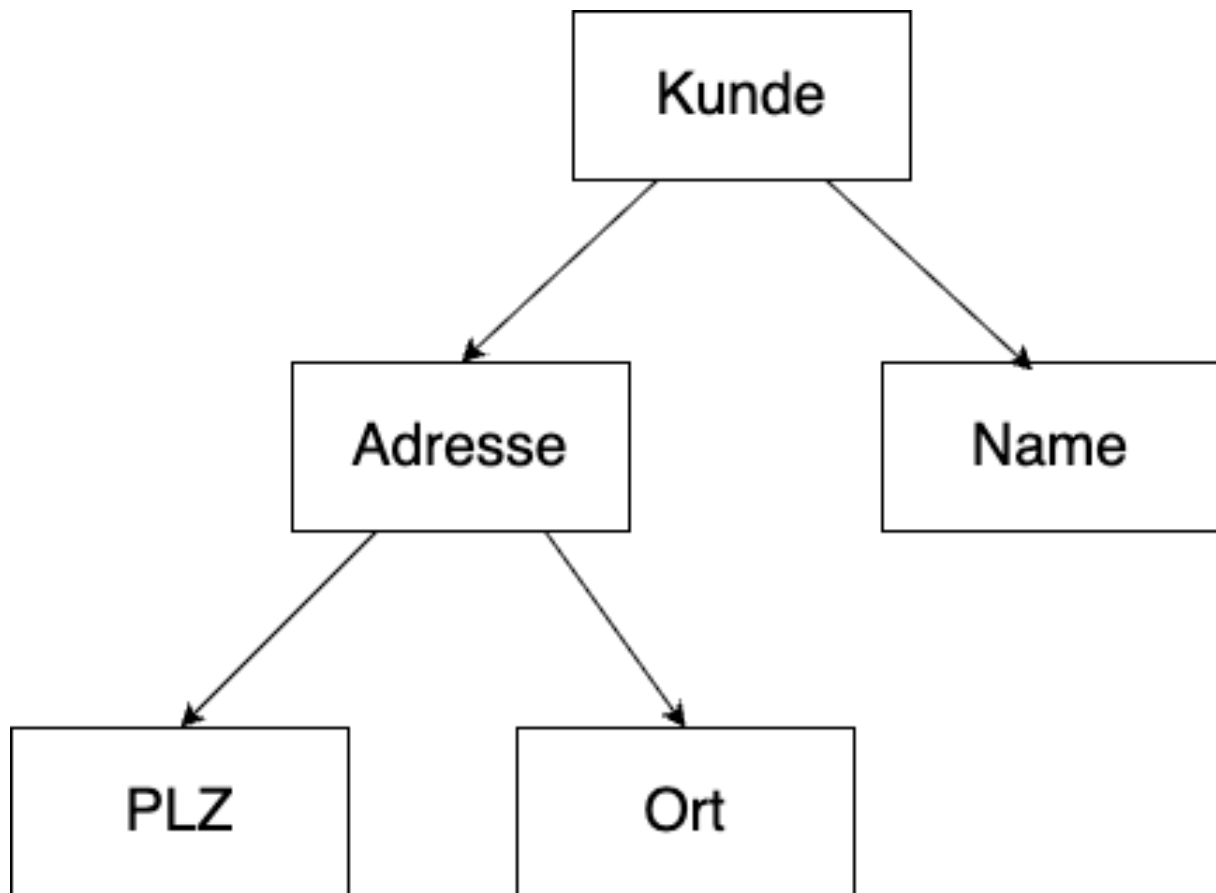


Abbildung 3. Beispiel: hierarchisches Datenbankmodell

Das hierarchische Modell erinnert an Baumstrukturen. Jede Entität hat einen Vorgänger – außer der ersten – und kann mehrere Nachfolger haben. Dadurch ist das Modell gut rekursiv durchsuchbar. Es gibt immer nur einen Einstiegspunkt.

Vorteile

- Effiziente computergerechte Datenorganisation
- Sehr schnell

Nachteile

- Benutzer muss die Datenstruktur gut kennen
- Redundanzen sind möglich

2.3 Netzwerkmodell

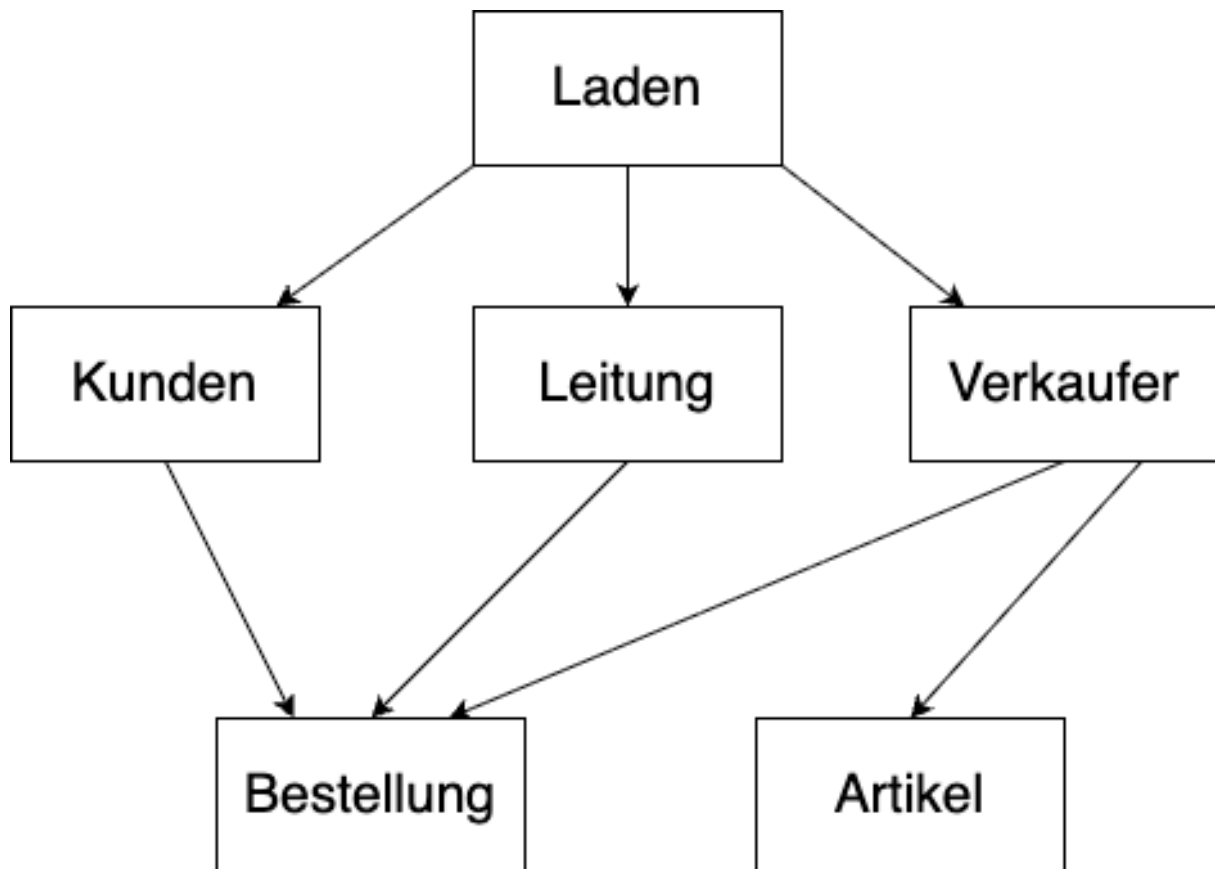


Abbildung 4. Beispiel: Netzwerk-Datenbankmodell

Das Netzwerkmodell ist nah am hierarchischen Modell, allerdings bietet es mehr als einen Einstiegspunkt. So kann man über einen Kunden an dessen Bestellung gelangen, aber auch über den Verkäufer.

Vorteile

- Flexibler Zugriff über mehrere Einstiegspunkte
- Komplexe Strukturen können abgebildet werden

Nachteile

- Benutzer muss die Datenstruktur gut kennen
- Unübersichtlich

2.4 Objektorientiertes Datenbankmodell

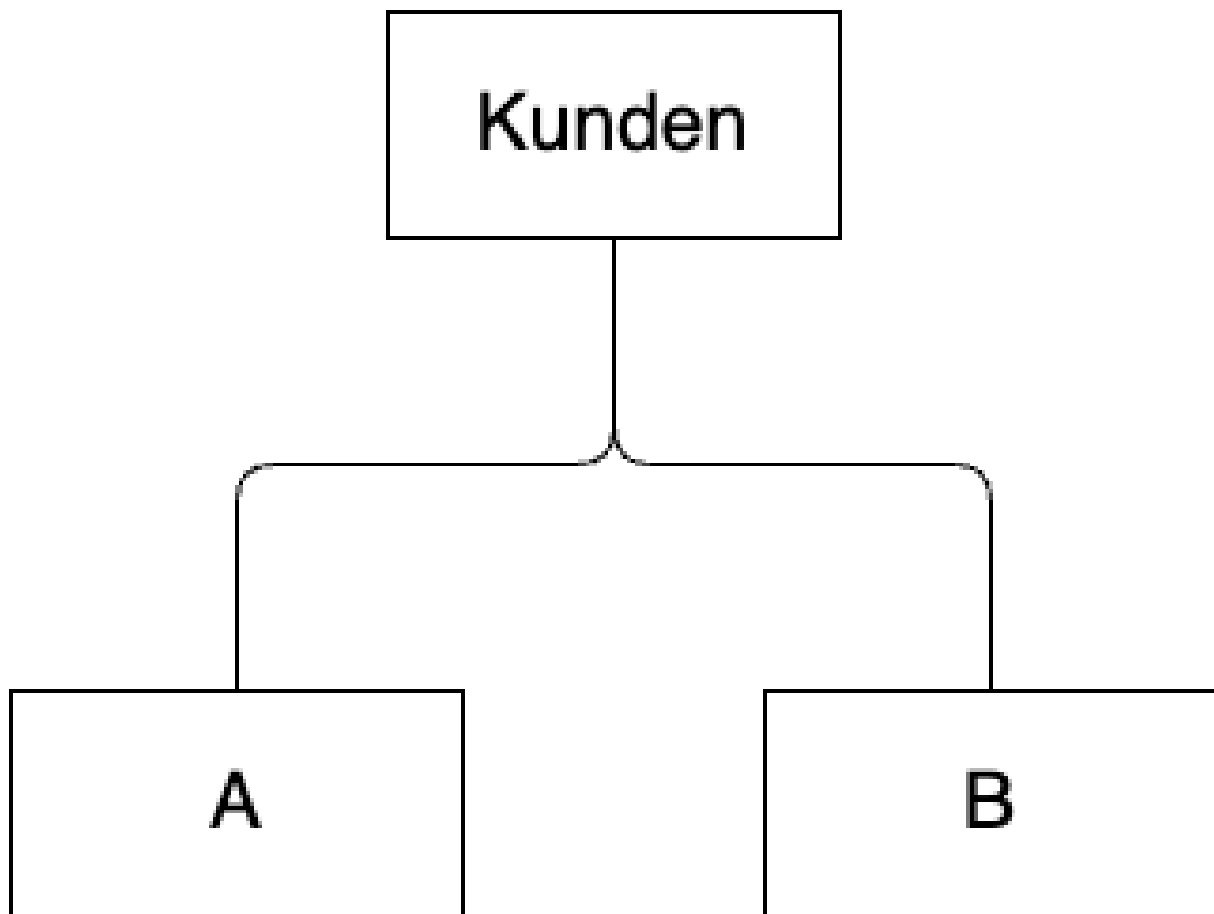


Abbildung 5. Beispiel: objektorientiertes Datenbankmodell

Bei einem objektorientierten Datenbankmodell werden Objekte ohne Zerlegung gespeichert, also als vollständige Objekte. Auch Vererbung, Kapselung und Polymorphie sind integriert. Dadurch lassen sich komplexe Datenstrukturen darstellen.

Heutzutage ist dieses Modell weitgehend obsolet, da es für relationale Datenbanken `ORM`s gibt, z. B. *Entity Framework Core* für C#.

Vorteile

- Das Problem des *Object-Relational Impedance Mismatch* wird behoben
- Komplexe Kompositionen (`JOIN`s) entfallen
- Schlüsselverwaltung systemintern

Nachteile

- Bis heute geringe Verbreitung (z. B. db4o), bei Mengenoperationen langsam
- Aufwändige Bearbeitung vererbter Objekte, inkl. Schreibsperrern

3 Datenverteilung

Logisch zusammenhängende Datenbestände werden physisch verteilt. Die Verwaltung erfolgt dabei übergeordnet durch das Datenbank-System (DB-System).

3.1 Gründe für die Datenverteilung

Die wesentlichen Gründe für die physische Verteilung von logisch zusammenhängenden Daten sind:

- **Verfügbarkeit:** Erhöhung der Systemverfügbarkeit durch Redundanz und lokale Zugriffsmöglichkeiten.
- **Sicherheit:** Schutz der Daten durch Verteilung und kontrollierte Zugriffsmöglichkeiten auf verschiedenen Knoten.
- **Effizienzsteigerungen:** Lokale Verarbeitung von Datenanfragen verringert die Netzwerklast und verbessert die Antwortzeiten.
- **Mobilität:** Unterstützung von mobilen Anwendungen und Zugriffen von verschiedenen geografischen Standorten aus.
- **Skalierbarkeit:** Einfachere Erweiterung der Systemkapazität durch Hinzufügen weiterer Knoten im verteilten System.
- **Zuverlässigkeit:** Erhöhte Ausfallsicherheit durch die Vermeidung von Single Points of Failure.

4 Datenbanken in verteilten Systemen

Verteilte Systeme

Ein verteiltes System ist eine Sammlung mehrerer unabhängiger Computer bzw. Prozesse, die nach außen hin wie ein einziges System wirken. Die einzelnen Knoten kommunizieren dabei über ein Netzwerk miteinander.

Bei verteilten Systemen und dem Zugriff auf eine Datenbank treten immer wieder Fehler auf. Häufige Problemfelder sind:

1. **Performance:** Bei verteilten Systemen kommen oft Performance-Probleme zum Tragen. Faktoren, die auf die Performance einwirken, sind die Leistung des Datenbankservers sowie der Code der Anwendung – etwa, ob ein `SQL`-Statement so geschrieben wurde, dass es die Daten effizient abfragt. Aber auch Up- und Download des Netzwerks sind wichtig. Gerade Mitarbeiter*innen, die über ein `VPN` im Netzwerk sind, erleben häufig eine starke Latenz. Bei vielen Anfragen an einen Datenbankserver verstärkt sich das noch, weil die Hardware an ihre Grenzen kommt.
2. **Verfügbarkeit:** Dadurch, dass ein Server meist viele Anfragen verarbeiten muss, ist eine Hochverfügbarkeit schwer zu erreichen. Daher wird oft darauf verzichtet, bestimmte Features in Programme einzubinden, da sonst der Server zu stark ausgelastet wird.
3. **Konsistenz:** Bei mehreren Nutzern ist es oft schwer, die Konsistenz zu halten – insbesondere, wenn mehrere Personen auf denselben Datensatz zugreifen. Beispiel: Eine Person aus dem Telefonverkauf greift auf einen Artikel zu, um dem Kunden den Preis zu nennen, während gleichzeitig eine Person aus dem Einkauf denselben Preis anpasst. Um die Konsistenz zu halten, werden oft einzelne Felder oder sogar ganze Datensätze gesperrt.

4.1 CAP-Theorem

Bevor wir uns Lösungen für diese Probleme ansehen, müssen wir das CAP-Theorem verstehen. Dieses wurde im Jahr 2000 von *Eric Brewer* aufgestellt und 2002 von *Seth Gilbert* und *Nancy Lynch* formal bewiesen. Das Theorem besagt, dass man Konsistenz (*Consistency*), Verfügbarkeit (*Availability*) und Partitionstoleranz (*Partition Tolerance*) nie gleichzeitig vollständig erreichen kann. Daher muss man immer abwägen, was wichtiger ist.

CAP-Theorem

Von den drei Eigenschaften Konsistenz, Verfügbarkeit und Partitionstoleranz sind in einem verteilten System stets nur zwei zugleich vollständig erreichbar.

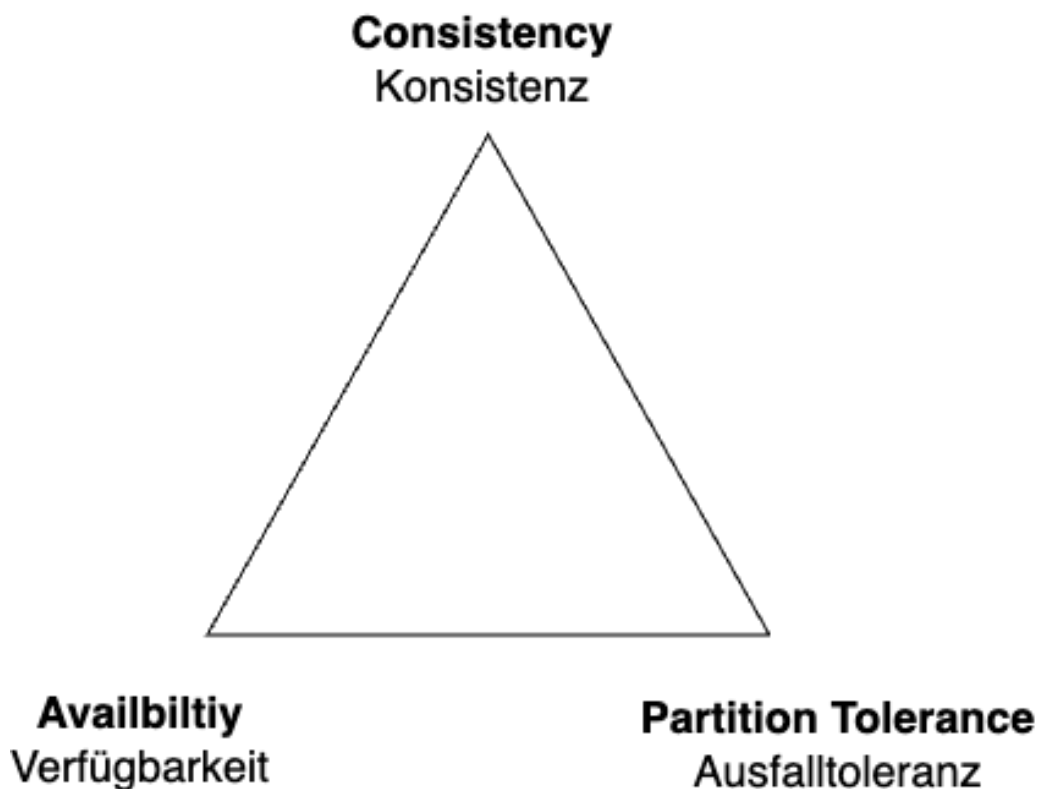


Abbildung 6. CAP-Theorem

4.2 Zentrale Datenbank

Bei einer zentralen Datenbank hat man einen zentralen `SQL`-Server, auf dem immer alle Abfragen laufen. Dadurch ist eine gewisse Konsistenz gewährleistet. Das Problem dabei ist jedoch, dass der `SQL`-Server ausgelastet werden kann, was sich auf die Performance auswirkt – dadurch wird alles langsamer, und auch die Ausfallsicherheit ist schwierig. Allerdings kann ein Server auch stark ausgestattet sein, sodass dies weniger ein Problem darstellt. Bei einem zentralen System muss man die Konfiguration vorab festlegen und kann später schlecht skalieren.

4.3 Replikation

Statt nur einen Server zu haben, nutzt man mehrere und repliziert die Daten. Damit hat man einen konsistenten Datenbestand, kann aber Anfragen und Last verteilen.

4.3.1 Arten

Primary / Secondary

Es gibt einen Hauptserver (*Primary*); die Replikationen werden auf andere Server kopiert (*Secondary*). Schreibzugriffe gibt es nur auf dem Primary, Leseanfragen können hingegen auch an die Secondary-Server geleitet werden.

Peer to Peer (Multi-Master)

Alle Nodes sind gleichberechtigt und erlauben Schreibzugriffe. Änderungen müssen daher immer auf allen Nodes synchronisiert werden.

4.3.2 Modus**Asynchrone Replikation**

Die Änderung ist sofort abgeschlossen und wird erst später repliziert. Die Performance der Schreiboperation ist gut, dafür ist der Datenbestand vorübergehend inkonsistent.

Synchrone Replikation

Die Änderung ist erst nach der Synchronisation abgeschlossen. Die Performance der Schreiboperation ist langsamer, dafür ist der Datenbestand durchgehend konsistent.

4.4 Sharding

Beim *Sharding* werden zusammengehörige Daten-Aggregate jeweils einem Node zugeordnet. Schreibzugriffe für einen Datensatz erfolgen immer nur über diesen einen, definierten Node. Dadurch sind Lese- und Schreiboperationen skalierbar, und durch die Daten-Lokalität ergibt sich eine bessere Latenz. Außerdem ist das System konsistent: Da Schreibzugriffe für einen Datensatz nur über einen Node laufen, entstehen keine Konflikte.

Sharding vs. Replikation

Bei der *Replikation* liegen dieselben Daten auf mehreren Nodes. Beim *Sharding* wird der Datenbestand aufgeteilt – jeder Node hält nur einen Teil der Daten.

Mittels Sharding können umfangreiche Datenmengen verwaltet werden, welche die Kapazitäten eines einzelnen Servers sprengen würden.

Nachteil

Der größte Nachteil des Shardings ist, dass ein Zugriff über andere Kriterien als das Aufteilungskriterium unverhältnismäßig aufwändig ist. Abfragen über andere Kriterien oder `JOINS` müssen im Allgemeinen auf alle Server aufgeteilt werden. Datenmodell und Zugriffspfade müssen daher so entworfen werden, dass derartige Zugriffe nur selten oder gar nicht vorkommen – sonst werden die Vorteile des Shardings zunichtegemacht.

Aufgrund dieser Eigenschaften ist Sharding ideal für `NoSQL`-Datenbanken.

4.5 Caching

Beim *Caching* verwalten die Clients eine eigene Kopie der Daten (oder von Teilen davon).

Vorteile

- Schneller, lokaler Zugriff
- Entlastung des Netzwerks und der Server

Nachteile

- Konsistenz – wie lange sind die gecachten Daten gültig?
- Bei Schreiboperationen im Cache sind Konflikte möglich

Beispiel: Caching in der Praxis

DNS, Browser-Cache, Offline-Webanwendungen sowie Microsoft BranchCache und Windows Updates.

4.6 Map/Reduce

Map/Reduce ist ein Programmiermodell zur Parallelisierung, das häufig in Software-Frameworks realisiert ist – etwa in Hadoop oder MongoDB. Komplexe Aufgaben werden dabei in zwei Phasen bearbeitet: *Map* und *Reduce*.

Divide and Conquer

Map/Reduce basiert auf dem Prinzip *Divide and Conquer* („Teile und herrsche“): Ein großes Problem wird rekursiv in kleinere, unabhängige Teilprobleme zerlegt, die einzeln gelöst werden. Anschließend werden die Teilergebnisse wieder zu einer Gesamtlösung zusammengefügt. Weil die Teilprobleme unabhängig voneinander sind, können sie parallel bearbeitet werden.

1 Map: Die Gesamtaufgabe wird in einzelne Arbeitspakete aufgeteilt und auf verschiedene Nodes verteilt. Jeder Node bearbeitet seine Teilaufgabe unabhängig von den anderen – dadurch ist eine parallele Bearbeitung möglich.

2 Reduce: Die Teilergebnisse der einzelnen Nodes werden wieder zu einem Gesamtergebnis zusammengefügt.

Kernidee

Durch das rekursive Aufteilen in unabhängige Teilaufgaben (*Divide and Conquer*) lässt sich die Bearbeitung über viele Nodes parallelisieren und damit auf großen Datenmengen skalieren.

5 Managementunterstützungssysteme

Hinter dem Begriff Managementunterstützungssystem steckt – für alle unter 50 – im Grunde *lobotomized Data Science*, also das Extrahieren von Wissen aus großen Datenmengen, nur ohne Betrachtung der Zukunft und mit einem stärkeren Fokus auf Berichte über vergangene Daten.

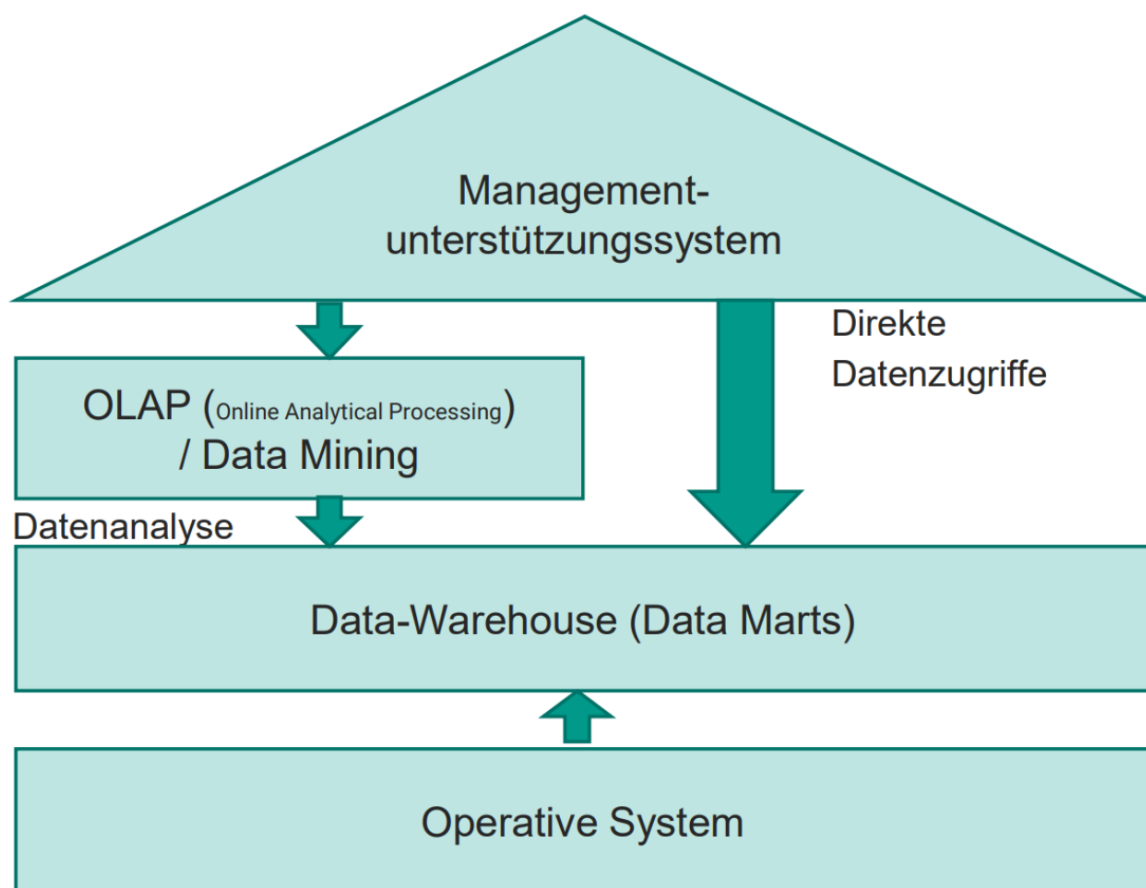


Abbildung 7. Aufbau eines Managementunterstützungssystems

5.1 Data Warehouse

Data Warehousing beschreibt ein Konzept zur Informationsanalyse, -selektion und -aufbereitung, bei dem dem Anwender entscheidungsrelevante Daten aus unterschiedlichen Quellen in einer einheitlichen, zentralen Systemumgebung zur Auswertung zur Verfügung gestellt werden.

- Ein Data Warehouse ist eine extrem große Datenbank, die unabhängig von den operativen Systemen installiert ist und ein vordefiniertes Set von meist verdichteten Daten enthält.
- Die Benutzer können auf die Datenbank beliebig zugreifen und alle gewünschten Daten herausziehen, sofern die Informationen enthalten sind und eine Zugriffsberechtigung vorliegt.
- Der Grad der Informationstiefe und die jeweilige Sichtweise können dabei indivi-

duell festgelegt werden.

- Betrachtungsebenen sind beispielsweise Konten, Kostenstellen, Kostenträger, Artikel, Kunden, Märkte oder Vertreter. Es gibt keine festen Vorgaben für die Art der Reihenfolge und der Verdichtung.

5.2 Data Mart

Data Marts sind abteilungs- oder funktionsbezogene Datenbanken mit einer speziellen Aufgabenstellung, die eine Untermenge der im zentralen Data Warehouse gespeicherten operativen Daten enthalten.

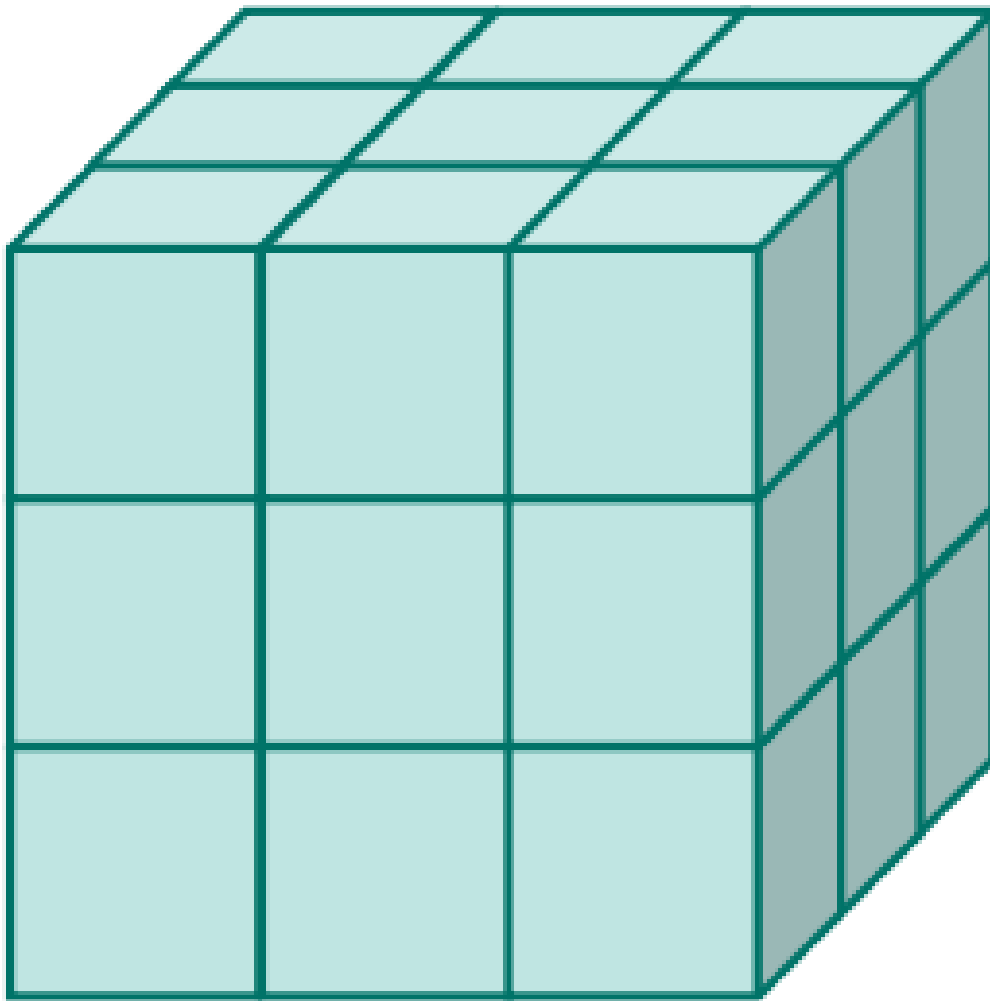


Abbildung 8. Data Mart

- Data Marts enthalten lediglich Informationen, die in einem bestimmten Funktionsbereich, z. B. Marketing, des Unternehmens zur Anwendung kommen.
- Data Marts bilden daher eine Teilmenge der umfassenden Data-Warehouse-Datenbank.
- Data Marts werden auch Datenwürfel, Power Cube oder Info Cube genannt.

5.3 Online Analytical Processing – OLAP

Online Analytical Processing ist ein „top-down“-Ansatz, um sehr flexibel mehrdimensionale Datenanalysen durchzuführen und Hypothesen zu bilden.

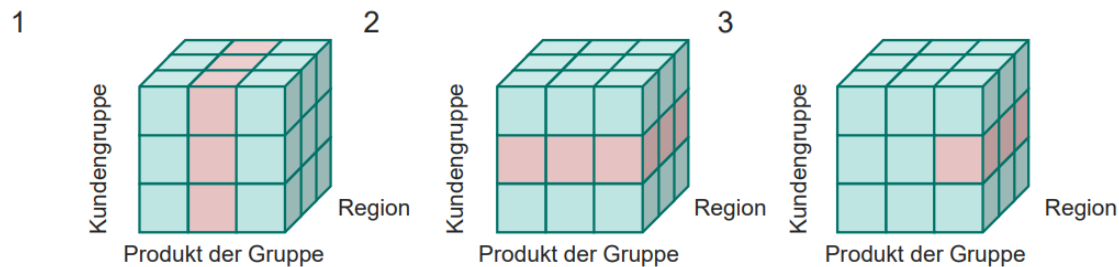


Abbildung 9. Datenanalyse mit OLAP

5.4 Data Mining

Data Mining bezeichnet das automatische Entdecken von Modellen und Mustern in großen Datenbeständen durch Anwendung bestimmter Data-Mining-Werkzeuge.

- Data Mining ist eine „intelligente“ Anwendung auf Basis einer Data-Warehouse-Architektur.
- Beziehungen zwischen Daten werden hergestellt (Zeitreihenanalyse, Datenklassifikation, Prognosen).
- Data Mining versucht, den Anwender auf bemerkenswerte Datenkonstellationen aufmerksam zu machen und zu signifikanten Aussagen hinzuführen.
- Analyseziel: „Finde Gold in deinen Daten!“

6 Datenbank-Architektur

6.1 ANSI-3-Ebenen-Modell

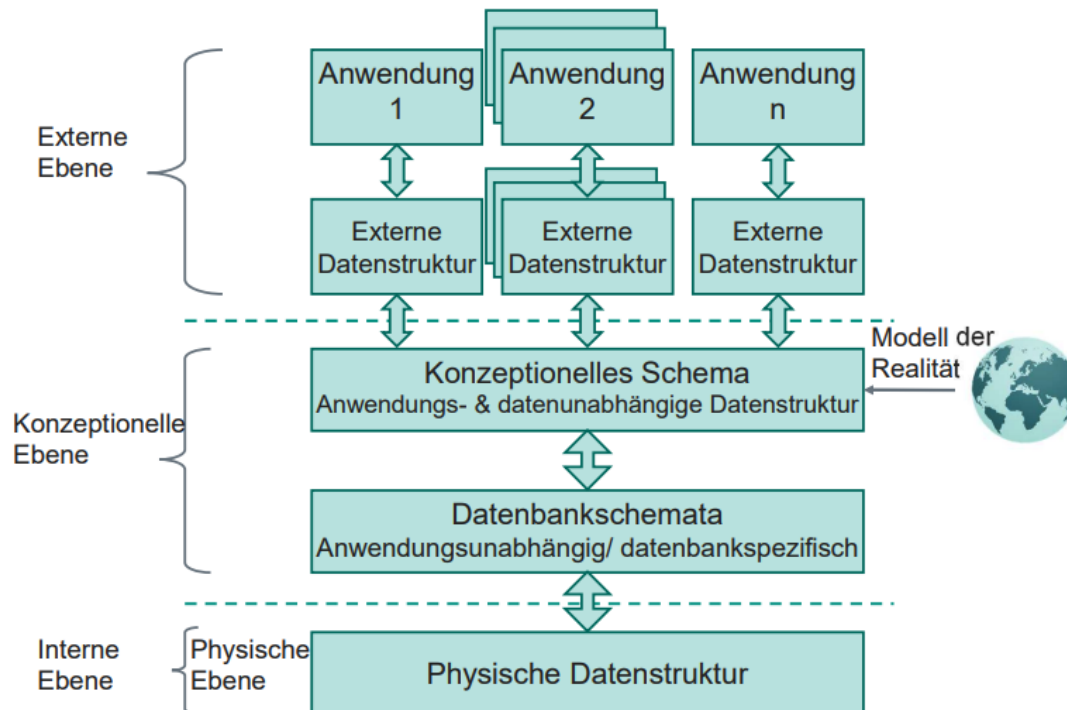


Abbildung 10. Aufbau des ANSI-3-Ebenen-Modells

Die ANSI-3-Ebenen-Architektur gibt es seit den 1970er-Jahren. *ANSI* steht für *American National Standards Institute* und ist damit in etwa mit dem deutschen DIN vergleichbar. Die 3-Ebenen-Architektur wird auch *SPARC* genannt, abgeleitet von *Standards Planning And Requirements Committee* – einem Ausschuss innerhalb von ANSI, von dem die Architektur stammt.

ANSI/SPARC-Architektur

Die 3-Sichten-Architektur ist die Grundlage einer konsistenten, modularen, interoperablen, wartbaren und skalierbaren Datenbank. Sie trennt die Daten in eine **interne**, eine **konzeptionelle** und eine **externe** Ebene.

6.1.1 Interne Ebene

Die interne Ebene ist das Fundament einer Datenbank. Hier liegt ein Datenbankmodell zugrunde, wie es im Kapitel *Datenbankmodelle* besprochen wird. Auf dieser Ebene werden die Aspekte der physischen Speicherung festgelegt – also *wie* und *wo* die Daten konkret abgelegt werden.

6.1.2 Konzeptionelle Ebene

Die konzeptionelle Ebene ist das Bindeglied zwischen der internen und der externen Ebene. Hier werden die Daten modelliert, beispielsweise über ein ER-Diagramm. Auf dieser

Ebene wird das Datenbankschema festgelegt, ebenso das konzeptionelle Schema für `Primary Key`, `Foreign Key` sowie sonstige Regeln zur Sicherung der Datenkonsistenz.

6.1.3 Externe Ebene

Die externe Ebene definiert, wie Nutzer auf die Datenbank zugreifen. Dazu zählen User Interfaces, Anwendungs-APIs, Views sowie – im Fall einer API – das Äquivalent in Form von *Data Transfer Objects* (`DTO`s). Hier können außerdem Plausibilitätsprüfungen und sonstige Integrationsregeln festgelegt werden. Auf dieser Ebene findet der `SQL`-Zugriff statt.

6.2 Datenbank-Entwurfsphasen

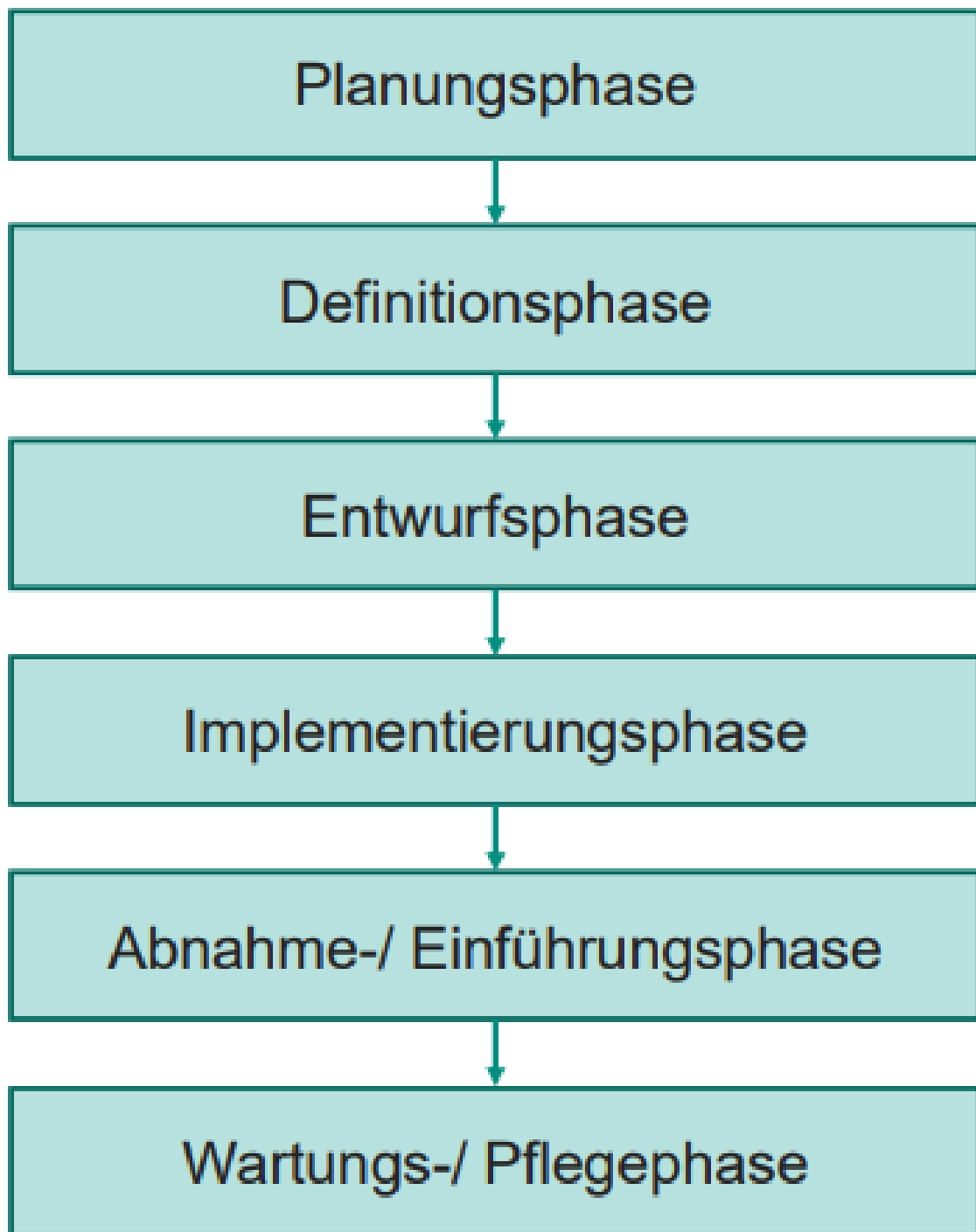


Abbildung 11. Entwurfsphasen einer Datenbank

Der Entwurf einer Datenbank lässt sich in sechs Phasen unterteilen.

6.2.1 Planungsphase

In der Planungsphase geht es um die grundlegenden Festlegungen – also darum, was überhaupt in der Datenbank gespeichert werden soll. Beispielsweise könnte ein Projekt darin bestehen, die Touren eines Logistikunternehmens zu speichern.

6.2.2 Definitionsphase

Hier wird festgelegt, welche Anforderungen die Datenbank erfüllen muss und welche Daten wo und wie erfasst werden müssen.

6.2.3 Entwurfsphase

Hier werden die technischen Entscheidungen getroffen – etwa, welches Datenbankmodell genutzt wird. Außerdem entsteht in dieser Phase das ER-Diagramm.

6.2.4 Implementierungsphase

Hier wird der Entwurf in Code umgesetzt, meist als `DDL` in `SQL`. Eventuell werden auch Views angelegt und vielleicht sogar eine API.

6.2.5 Abnahme- und Einführungsphase

Hier werden die Datenbank bzw. die umgebende Software eingeführt. Anschließend wird geprüft, ob die Anwender mit der Anwendung zufrieden sind.

6.2.6 Wartungs- und Pflegephase

Hier werden mögliche Fehler behoben; auch eventuelle Erweiterungen fallen in diese Phase.

7 NoSQL

NoSQL steht für *Not Only SQL* und bietet eine Alternative zu relationalen Datenbank-Management-Systemen.

Das liegt insbesondere daran, dass relationale Datenbanksysteme bei flexiblen Datenstrukturen (etwa für Konfigurationen), hoher Schreiblast, globalen Netzwerken und sehr großen Datenmengen an ihre Grenzen kommen - gerade bei Echtzeitanalysen oder **SaaS**-Anwendungen. Auch wenn man an die 5 V von Big Data denkt, ist NoSQL für Varietät wichtig.

Eine andere Sache, die man bei NoSQL im Kopf behalten sollte, ist, dass eine NoSQL-Datenbank oder eine relationale Datenbank nie die wirkliche Lösung ist, sondern man für unterschiedliche Daten unterschiedliche Datenbanksysteme nutzt.

Die Ziele und die Motivation von NoSQL sind:

1. ein einfacheres Design
2. einfache Skalierung
3. Kontrolle über die Daten
4. verteilte Architektur – eine passende Datenbank für jeden Use Case
5. größere Datenbanken für Big Data
6. horizontale Skalierung in der Cloud
7. flexible Datenstruktur
8. hohe Schreiblast

7.1 Modelle

7.1.1 Key-Value

In einem Key-Value-Store – wie beispielsweise einem **Redis**-Cache – werden Daten als Schlüssel-Wert-Paare (*Key-Value-Pairs*) gespeichert. Ein Key-Value-Modell eignet sich gut für temporäre Daten.

7.1.2 Document Store

Daten werden in Dokumenten gespeichert. Ein Beispiel ist **MongoDB**, das mit **JSON**-Dateien arbeitet. MongoDB ist besonders gut für Konfigurationsdateien geeignet.

7.1.3 Column Store

Bei einem Column-Store existieren keine festen Schemas, da jede Zeile andere Spalten besitzen kann.

7.1.4 Graphdatenbank

Eine Graphdatenbank stellt Daten als Graphen dar. Die Verknüpfung über Knoten ermöglicht die Darstellung von Netzwerken und Wortzusammenhängen und bildet so die Grundlage für **LLMs** und Big Data. Ziel ist es, Relationen abzubilden.